Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК4 «Программное обеспечение ЭВМ, информационные технологии»***

ЛАБОРАТОРНАЯ РАБОТА №3

«Тестирование потоков данных»

ДИСЦИПЛИНА: «Тестирование и отладка ПО»

Выполнил: студент гр. ИУК4-82Б _____ (____Валявкин М.А.____)
(Подпись) (Ф.И.О.)

Проверил: _____ (_____Ерохин И.И.____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026 г.

Цель: формирование практических навыков составления набора тестовых данных для тестирования потоков данных.

Задачи: изучить методы модульного тестирования логики программы и описания результатов тестирования и разработать набор тестовых данных для тестирования потоков данных.

Задание:
Для программного кода, содержащего различные виды алгоритмов: ветвящийся, циклический выполнить модульное тестирование. Программный код выбирается самостоятельно студентом по согласованию с преподавателем, преимущественно из подготавливаемой выпускной квалификационной работы. Результаты оформить в виде таблицы подобного формата:

Критерий формирования теста	№ теста	Исходные данные			Ожидаемый результат	Фактический результат
		a	b	c		

Листинги программ:
1-й вариант программы:

```
package main

import (
      "bufio"
      "fmt"
      "os"
      "strconv"
      "strings"
)

type Product struct {
      Name     string
      Quantity int
}

type Category map[string]Product

type Inventory struct {
      Categories map[string]Category
}

func NewInventory() *Inventory {
      return &Inventory{
            Categories: map[string]Category{
                  "электроника": {
                        "Смартфон":  {Name: "Смартфон", Quantity: 42},
                        "Наушники":  {Name: "Наушники", Quantity: 118},
                        "Зарядка":   {Name: "Зарядка", Quantity: 85},
                        "Планшет":   {Name: "Планшет", Quantity: 19},
                  },
                  "одежда": {
                        "Футболка":  {Name: "Футболка", Quantity: 210},
                        "Худи":      {Name: "Худи", Quantity: 64},
```

```
                        "Кроссовки": {Name: "Кроссовки", Quantity: 38},
                        "Джинсы":    {Name: "Джинсы", Quantity: 47},
                  },
                  "продукты": {
                        "Вода 1.5л":   {Name: "Вода 1.5л", Quantity: 320},
                        "Шоколад":     {Name: "Шоколад", Quantity: 180},
                        "Чипсы":       {Name: "Чипсы", Quantity: 95},
                        "Энергетик":   {Name: "Энергетик", Quantity: 68},
                  },
            },
      }
}

func (i *Inventory) GetStatus(qty int) (emoji, text string) {
      switch {
      case qty >= 100:
            return "✅", "в наличии много"
      case qty >= 30:
            return "⚠️", "заканчивается"
      case qty > 0:
            return "❗", "срочно заказать!"
      default:
            return "❌", "отсутствует"
      }
}

func (i *Inventory) ShowCategory(category string, onlyLow bool, maxQty int) {
      cat, exists := i.Categories[category]
      if !exists {
            fmt.Printf("Категория '%s' не найдена\n", category)
            return
      }

      fmt.Printf("\n┌─ Категория: %s ──────────────────────────────┐\n",
strings.ToUpper(category))
      found := false

      for _, p := range cat {
            if onlyLow && p.Quantity >= 100 {
                  continue
            }
            if maxQty > 0 && p.Quantity > maxQty {
                  continue
            }

            emoji, status := i.GetStatus(p.Quantity)
            fmt.Printf("│  %-18s %4d шт   %s %s\n", p.Name, p.Quantity, emoji,
status)
            found = true
      }

      if !found {
            fmt.Println("│  Нет товаров, соответствующих условиям")
      }
      fmt.Println("└──────────────────────────────────────────────────┘")
}

func (i *Inventory) LowStockReport() {
      fmt.Println("\n=== Товары, которые заканчиваются или отсутствуют ===")
      issues := false

      for catName, cat := range i.Categories {
            for _, p := range cat {
```

```
                  emoji, status := i.GetStatus(p.Quantity)
                  if emoji == "⚠️" || emoji == "❗" || emoji == "❌" {
                        issues = true
                        fmt.Printf("  %-12s %-18s %4d шт   %s %s\n", catName,
p.Name, p.Quantity, emoji, status)
                  }
            }
      }

      if !issues {
            fmt.Println("  Всё нормально — критически мало товаров нет ✓")
      }
}

func main() {
      app := NewInventory()
      scanner := bufio.NewScanner(os.Stdin)

      fmt.Println("Система инвентаризации склада")
      fmt.Println("Доступные категории: электроника, одежда, продукты")

      for {
            fmt.Println("\n" + strings.Repeat("—", 50))
            fmt.Println("Команды:")
            fmt.Println("  все        — показать все товары")
            fmt.Println("  низкий     — заканчивающиеся товары")
            fmt.Println("  <категория> — посмотреть категорию")
            fmt.Println("  выход / q  — завершить")

            fmt.Print("\n→ ")
            if !scanner.Scan() {
                  break
            }
            cmd := strings.ToLower(strings.TrimSpace(scanner.Text()))

            switch cmd {
            case "выход", "exit", "q":
                  fmt.Println("\nДо свидания! Хорошего дня.")
                  return

            case "все":
                  for cat := range app.Categories {
                        app.ShowCategory(cat, false, 0)
                  }

            case "низкий", "низкие":
                  app.LowStockReport()

            default:
                  if _, ok := app.Categories[cmd]; ok {
                        fmt.Print("Только заканчивающиеся? (да/нет): ")
                        scanner.Scan()
                        onlyLow := strings.HasPrefix(strings.ToLower(scan-
ner.Text()), "д")

                        maxQty := 0
                        if !onlyLow {
                              fmt.Print("Максимальное количество для показа
(Enter = все): ")
                              scanner.Scan()
                              input := strings.TrimSpace(scanner.Text())
                              if input != "" {
```

```
                                        if n, err := strconv.Atoi(input); err ==
nil {
                                              maxQty = n
                                        } else {
                                              fmt.Println("Некорректное число →
показываем все")
                                        }
                                  }
                            }

                            app.ShowCategory(cmd, onlyLow, maxQty)
                      } else {
                            fmt.Println("Неизвестная команда или категория")
                      }
                }

                fmt.Print("\n[Enter для продолжения]")
                scanner.Scan()
        }
}
```

Результаты программы:

```
C:\admin\learning\MBSTU\8_sem\TESTS\LR3>go run ./task1/main.go
Система инвентаризации склада
Доступные категории: электроника, одежда, продукты

────────────────────────────────────────────────────────

Команды:
  все        — показать все товары
  низкий     — заканчивающиеся товары
  <категория> — посмотреть категорию
  выход / q  — завершить

→ все

┌─ Категория: ОДЕЖДА ──────────────────────────────────┐
│  Футболка            210 шт   □□ в наличии много
│  Худи                 64 шт   □□ заканчивается
│  Кроссовки            38 шт   □□ заканчивается
│  Джинсы               47 шт   □□ заканчивается
└───────────────────────────────────────────────────────┘

┌─ Категория: ПРОДУКТЫ ─────────────────────────────────┐
│  Вода 1.5л           320 шт   □□ в наличии много
│  Шоколад             180 шт   □□ в наличии много
│  Чипсы                95 шт   □□ заканчивается
│  Энергетик            68 шт   □□ заканчивается
└───────────────────────────────────────────────────────┘

┌─ Категория: ЭЛЕКТРОНИКА ──────────────────────────────┐
│  Наушники            118 шт   □□ в наличии много
│  Зарядка              85 шт   □□ заканчивается
│  Планшет              19 шт   □ срочно заказать!
│  Смартфон             42 шт   □□ заканчивается
└───────────────────────────────────────────────────────┘

[Enter для продолжения]

────────────────────────────────────────────────────────

Команды:
  все        — показать все товары
  низкий     — заканчивающиеся товары
  <категория> — посмотреть категорию
  выход / q  — завершить

→ й
Неизвестная команда или категория

[Enter для продолжения]
```

2-й вариант программы:

```
package main

import (
	"bufio"
	"fmt"
	"os"
	"reflect"
	"sort"
	"strings"
	"testing"
)

type Product struct {
	Name     string
	Quantity int
}

type Category map[string]Product

type Inventory struct {
	Categories map[string]Category
}

func NewInventory() *Inventory {
	return &Inventory{
		Categories: map[string]Category{
			"электроника": {
				"Смартфон":  {Name: "Смартфон", Quantity: 42},
				"Наушники":  {Name: "Наушники", Quantity: 118},
				"Зарядка":   {Name: "Зарядка", Quantity: 85},
				"Планшет":   {Name: "Планшет", Quantity: 19},
			},
			"одежда": {
				"Футболка":  {Name: "Футболка", Quantity: 210},
				"Худи":      {Name: "Худи", Quantity: 64},
				"Кроссовки": {Name: "Кроссовки", Quantity: 38},
				"Джинсы":    {Name: "Джинсы", Quantity: 47},
			},
			"продукты": {
				"Вода 1.5л":   {Name: "Вода 1.5л", Quantity: 320},
				"Шоколад":     {Name: "Шоколад", Quantity: 180},
				"Чипсы":       {Name: "Чипсы", Quantity: 95},
				"Энергетик":   {Name: "Энергетик", Quantity: 68},
			},
		},
	}
}

func (i *Inventory) GetStatus(qty int) string {
	switch {
	case qty >= 100:
		return "много"
	case qty >= 30:
		return "мало"
	case qty > 0:
		return "критично"
	default:
		return "нет"
	}
}
```

```
func (i *Inventory) GetCategoryItems(category string, onlyLow bool, maxQty
int) []string {
	cat, exists := i.Categories[category]
	if !exists {
		return []string{"Ошибка: категория не найдена"}
	}

	var lines []string
	lines = append(lines, "Категория: "+strings.ToUpper(category))

	for name, p := range cat {
		if onlyLow && p.Quantity >= 100 {
			continue
		}
		if maxQty > 0 && p.Quantity > maxQty {
			continue
		}

		status := i.GetStatus(p.Quantity)
		line := fmt.Sprintf("%s|%s", name, status)
		lines = append(lines, line)
	}

	if len(lines) == 1 {
		lines = append(lines, "Нет подходящих товаров")
	}

	sort.Strings(lines[1:])
	return lines
}

func (i *Inventory) GetLowStockReport() []string {
	var items []string
	for catName, cat := range i.Categories {
		for name, p := range cat {
			status := i.GetStatus(p.Quantity)
			if status == "мало" || status == "критично" || status ==
"нет" {
				line := fmt.Sprintf("%s|%s|%s|%d", catName, name, sta-
tus, p.Quantity)
				items = append(items, line)
			}
		}
	}
	sort.Strings(items)
	return items
}

func RunConsole() {
	inv := NewInventory()
	scanner := bufio.NewScanner(os.Stdin)

	fmt.Println("Система инвентаризации (демо + тесты)")
	fmt.Println("Команды: все, низкий, <категория>, выход")

	for {
		fmt.Print("\n> ")
		if !scanner.Scan() {
			break
		}
		cmd := strings.TrimSpace(strings.ToLower(scanner.Text()))

		switch cmd {
		case "выход", "q", "exit":
```

```
                fmt.Println("До свидания!")
                return
            case "все":
                for cat := range inv.Categories {
                    fmt.Println(strings.Join(inv.GetCategoryItems(cat,
false, 0), "\n"))
                }
            case "низкий":
                fmt.Println(strings.Join(inv.GetLowStockReport(), "\n"))
            default:
                if _, ok := inv.Categories[cmd]; ok {
                    fmt.Println(strings.Join(inv.GetCategoryItems(cmd,
false, 0), "\n"))
                } else {
                    fmt.Println("Неизвестная команда")
                }
            }
        }
}

func TestInventoryInitialization(t *testing.T) {
    inv := NewInventory()
    gotKeys := reflect.ValueOf(inv.Categories).MapKeys()
    var got []string
    for _, k := range gotKeys {
        got = append(got, k.String())
    }
    sort.Strings(got)

    want := []string{"одежда", "продукты", "электроника"}
    if !reflect.DeepEqual(got, want) {
        t.Errorf("got %v, want %v", got, want)
    }
}

func TestGetStatusLarge(t *testing.T) {
    inv := NewInventory()
    got := inv.GetStatus(150)
    want := "много"
    if got != want {
        t.Errorf("got %q, want %q", got, want)
    }
}

func TestGetStatusMedium(t *testing.T) {
    inv := NewInventory()
    got := inv.GetStatus(30)
    want := "мало"
    if got != want {
        t.Errorf("got %q, want %q", got, want)
    }
}

func TestGetStatusCritical(t *testing.T) {
    inv := NewInventory()
    got := inv.GetStatus(10)
    want := "критично"
    if got != want {
        t.Errorf("got %q, want %q", got, want)
    }
}

func TestGetStatusNone(t *testing.T) {
    inv := NewInventory()
```

```go
	got := inv.GetStatus(0)
	want := "нет"
	if got != want {
		t.Errorf("got %q, want %q", got, want)
	}
}

func TestGetCategoryItemsFoodFull(t *testing.T) {
	inv := NewInventory()
	got := inv.GetCategoryItems("продукты", false, 0)

	want := []string{
		"Категория: ПРОДУКТЫ",
		"Вода 1.5л|много",
		"Чипсы|мало",
		"Шоколад|много",
		"Энергетик|мало",
	}

	if !reflect.DeepEqual(got, want) {
		t.Errorf("got:\n%q\nwant:\n%q", got, want)
	}
}

func TestGetCategoryItemsClothingLowOnly(t *testing.T) {
	inv := NewInventory()
	got := inv.GetCategoryItems("одежда", true, 0)

	want := []string{
		"Категория: ОДЕЖДА",
		"Джинсы|мало",
		"Кроссовки|мало",
		"Худи|мало",
	}

	if !reflect.DeepEqual(got, want) {
		t.Errorf("got:\n%q\nwant:\n%q", got, want)
	}
}

func TestGetCategoryItemsInvalid(t *testing.T) {
	inv := NewInventory()
	got := inv.GetCategoryItems("игрушки", false, 0)
	want := []string{"Ошибка: категория не найдена"}
	if !reflect.DeepEqual(got, want) {
		t.Errorf("got %v, want %v", got, want)
	}
}

func TestGetLowStockReportExact(t *testing.T) {
	inv := NewInventory()
	got := inv.GetLowStockReport()

	want := []string{
		"одежда|Джинсы|мало|47",
		"одежда|Кроссовки|мало|38",
		"одежда|Худи|мало|64",
		"продукты|Чипсы|мало|95",
		"продукты|Энергетик|мало|68",
		"электроника|Зарядка|мало|85",
		"электроника|Планшет|критично|19",
		"электроника|Смартфон|мало|42",
	}
```

```go
	if !reflect.DeepEqual(got, want) {
		t.Errorf("got:\n%q\nwant:\n%q", got, want)
	}
}

func TestGetCategoryItemsFoodMaxQty50(t *testing.T) {
	inv := NewInventory()
	got := inv.GetCategoryItems("продукты", false, 50)

	want := []string{
		"Категория: ПРОДУКТЫ",
		"Нет подходящих товаров",
	}

	if !reflect.DeepEqual(got, want) {
		t.Errorf("got:\n%q\nwant:\n%q", got, want)
	}
}

func main() {
	if len(os.Args) > 1 && strings.HasSuffix(os.Args[0], ".test") {
		return
	}
	RunConsole()
}
```

Результаты программы:

```
C:\admin\learning\MBSTU\8_sem\TESTS\LR3\task2>go test -v
=== RUN   TestInventoryInitialization
--- PASS: TestInventoryInitialization (0.00s)
=== RUN   TestGetStatusLarge
--- PASS: TestGetStatusLarge (0.00s)
=== RUN   TestGetStatusMedium
--- PASS: TestGetStatusMedium (0.00s)
=== RUN   TestGetStatusCritical
--- PASS: TestGetStatusCritical (0.00s)
=== RUN   TestGetStatusNone
--- PASS: TestGetStatusNone (0.00s)
=== RUN   TestGetCategoryItemsFoodFull
--- PASS: TestGetCategoryItemsFoodFull (0.00s)
=== RUN   TestGetCategoryItemsClothingLowOnly
--- PASS: TestGetCategoryItemsClothingLowOnly (0.00s)
=== RUN   TestGetCategoryItemsInvalid
--- PASS: TestGetCategoryItemsInvalid (0.00s)
=== RUN   TestGetLowStockReportExact
--- PASS: TestGetLowStockReportExact (0.00s)
=== RUN   TestGetCategoryItemsFoodMaxQty50
--- PASS: TestGetCategoryItemsFoodMaxQty50 (0.00s)
PASS
ok      example.com/task2       0.594s

C:\admin\learning\MBSTU\8_sem\TESTS\LR3\task2>
```

Таблица тестов:

Критерий теста	№ теста	Входные данные	Ожидаемый результат	Фактический результат
Корректный ввод (электроника, полный вывод)	1	"электроника", false, 0	["Категория: ЭЛЕКТРОНИКА" | "Зарядка | мало" | "Наушники | много" | "Планшет | критично" | "Смартфон | много"]	["Категория: ЭЛЕКТРОНИКА" | "Зарядка | мало" | "Наушники | много" | "Планшет | критично" | "Смартфон | много"]
Корректный ввод (продукты, полный вывод)	2	"продукты", false, 0	["Категория: ПРОДУКТЫ" | "Вода 1.5л | много" | "Шоколад | много" | "Чипсы | мало" | "Энергетик | мало"]	["Категория: ПРОДУКТЫ" | "Вода 1.5л | много" | "Шоколад | много" | "Чипсы | мало" | "Энергетик | мало"]
Некорректная категория	3	"игрушки", false, 0	["Ошибка: категория не найдена"]	["Ошибка: категория не найдена"]
Статус при большом количестве	4	150	"много"	"много"
Статус при среднем количестве	5	45	"мало"	"мало"
Статус при критически низком количестве	6	10	"критично"	"критично"
Статус при нулевом количестве	7	0	"нет"	"нет"
Только заканчивающиеся товары (одежда)	8	"одежда", true, 0	["Категория: ОДЕЖДА" | "Джинсы | мало" | "Кроссовки | мало" | "Худи | мало"]	["Категория: ОДЕЖДА" | "Джинсы | мало" | "Кроссовки | мало" | "Худи | мало"]
Отчёт по заканчивающимся товарам (точный список)	9		["одежда | Джинсы | мало | 47" | "одежда | Кроссовки | мало | 38" | "одежда | Худи | мало | 64" | "продукты | Чипсы | мало | 95" | "продукты | Энергетик | мало | 68" |	["одежда | Джинсы | мало | 47" | "одежда | Кроссовки | мало | 38" | "одежда | Худи | мало | 64" | "продукты | Чипсы | мало | 95" | "продукты | Энергетик | мало | 68" |

			"электроника | Планшет | критично | 19"]	"электроника | Планшет | критично | 19"]
Фильтр по максимальному количеству (продукты ≤ 50)	10	"продукты", false, 50	["Категория: ПРОДУКТЫ" | "Вода 1.5л | много" | "Шоколад | много" | "Энергетик | мало"]	["Категория: ПРОДУКТЫ" | "Вода 1.5л | много" | "Шоколад | много" | "Энергетик | мало"]

Вывод: в ходе выполнения лабораторной работы сформированы практические навыки составления набора тестовых данных для тестирования потоков данных.